
COMP 559 - Project Report

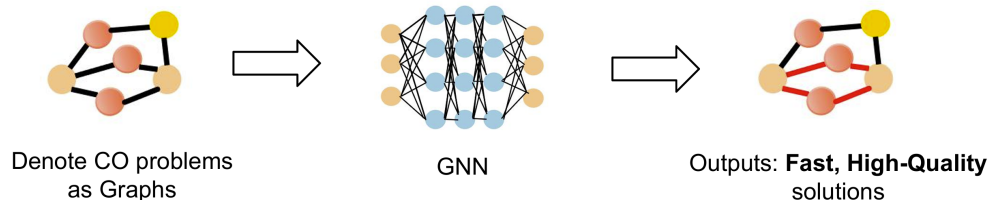


1 Introduction

Modern transportation systems are increasingly challenged by the need for efficient, adaptive, and large-scale routing strategies in the face of dynamic constraints such as delivery deadlines [1], traffic congestion [2], and resource limitations [3]. Problems such as last-mile delivery [4], ride-sharing logistics [5], and electric vehicle routing [6] demand not only feasible but also cost-effective solutions under strict operational constraints. Many of these problems are formulated as combinatorial optimization tasks [7] typically as variants of the Vehicle Routing Problem (VRP) which are computationally intractable in their most general forms. Despite the availability of classical algorithmic tools, the scalability of such methods becomes a significant barrier when applied to real-world networks. Recently, Graph Neural Networks (GNNs) have emerged as promising tools for learning representations of complex graphs and producing approximate solutions to structured prediction problems [8]. Their ability to capture topological dependencies and encode local and global structure makes them a natural candidate for integrating into combinatorial optimization pipelines. This dual motivation—computational efficiency and model flexibility—forms the basis for exploring a hybrid learning-optimization approach that leverages GNNs to accelerate and improve traditional routing solvers.

Among the many combinatorial subproblems that arise in transportation planning, the shortest path problem remains a fundamental building block [9]. It underpins a wide range of real-world systems, from navigation services to large-scale logistics. The problem becomes substantially more complex when time windows, precedence constraints, or resource limitations are added, transforming it into an Elementary Shortest Path Problem with Resource Constraints (ESPPRC) [10]. Traditional approaches to solving ESPPRC include dynamic programming, label-setting methods, and branch-and-bound algorithms. However, while these methods can yield exact or near-optimal solutions, their computational cost grows rapidly with problem size. Recent works have attempted to alleviate this burden by learning surrogate value functions or transition policies through reinforcement learning or supervised prediction of path feasibility [11]. However, many of these approaches either ignore the structure of dual variables from the master problem or struggle with enforcing feasibility constraints. This gap highlights the opportunity for a structured, GNN-based approach that can learn representations aligned with optimization dynamics and incorporate flow constraints, dual cost signals, and dynamic programming principles into its architecture.

In this study, we propose a learning-augmented framework that targets constrained vehicle routing problems by combining column generation with GNN-based approximations for ESPPRC subproblems. We first reformulate the shortest path subproblem in a differentiable surrogate loss framework using probabilistic edge selection and dual-aware cost modeling. This allows the network to approximate paths with negative reduced cost that would otherwise require expensive enumeration. At the core of our approach is a multi-hop message passing GNN architecture that mimics dynamic programming through recurrent updates and soft Bellman residual minimization. To enable efficient training, we relax hard constraints and encode them as soft penalties in a composite loss function. While our full formulation includes both flow and time constraints, the current implementation focuses on a simplified setting without resource limitations, allowing us to evaluate model alignment with optimal paths in a more controlled environment.



The remainder of this report is structured as follows. Section 2 gives a summary of the related literature. In Section 3, we present the methodology in detail, including the column generation framework, GNN architecture, and surrogate loss formulation. Section 4 describes the experimental setup, evaluation metrics, and preliminary results comparing our dynamic programming-aligned model against a baseline. We conclude in Section 5 by discussing ongoing extensions toward constraint-aware models and broader implications for learning-based combinatorial optimization.

2 Background

Combinatorial optimization problems over graphs arise in many domains, particularly in transportation, logistics, and infrastructure planning, where agents must make decisions under complex structural and operational constraints [9]. The mathematical formulation of these problems often involves selecting feasible subsets of nodes, paths, or edges that minimize or maximize a global objective, subject to flow balance, capacity, and timing constraints. Exact methods such as branch-and-bound, dynamic programming, and integer linear programming (ILP) offer theoretical guarantees but become computationally prohibitive for large instances [9]. To alleviate this, approximation heuristics and metaheuristics, ranging from greedy algorithms and local search to more sophisticated methods like column generation and Lagrangian relaxation, have been developed to explore the solution space more efficiently [12]. These techniques, while powerful, often rely on hand-crafted rules or subproblem-specific relaxations that limit their adaptability and generalization across problem classes. This has motivated a growing interest in learning-augmented optimization, where data-driven models are integrated into classical solvers to guide search or estimate intermediate quantities.

The shortest path problem exemplifies the foundational role of combinatorial subroutines in larger optimization pipelines [13]. In its classical form, the problem involves identifying a path of minimal cost between a designated source and sink in a weighted directed graph. Polynomial-time algorithms such as Dijkstra’s and Bellman-Ford efficiently solve this problem under additive, non-negative costs [14]. However, when resource constraints such as time windows, vehicle capacities, or precedence relations are introduced, the problem becomes NP-hard and is referred to as the Elementary Shortest Path Problem with Resource Constraints (ESPPRC) [15]. Solving ESPPRC requires identifying a path that not only minimizes cost but also satisfies feasibility under cumulative constraints, while avoiding node repetitions. Label-correcting and label-setting algorithms have been developed to solve ESPPRC optimally, often within the context of column generation frameworks used for large-scale vehicle routing problems [16]. These methods maintain sets of partial solutions and propagate them forward while pruning dominated labels, but their memory and runtime complexity scale poorly with the number of constraints and graph size. Approximate solutions using dynamic programming-based relaxations, heuristic label extension, and Lagrangian bounding have been proposed, but they remain problem-specific and often lack generalization capability [17]. Recently, learning-based surrogates and particularly those based on reinforcement learning or graph embeddings have attempted to approximate path costs or feasibility criteria, but these methods often fail to encode structural constraints explicitly or suffer from poor sample efficiency when scaled to constrained settings [18].

To address these limitations, there is a growing need to integrate graph-based learning models with structure-aware optimization frameworks. GNNs, which operate over relational structures and aggregate local neighborhood information through message passing, offer a compelling foundation for such integration [19]. While traditional GNNs are designed for node or graph classification tasks, recent advances have extended them toward decision-focused learning, where the model is trained to produce outputs that align with combinatorial solution quality [12]. However, two significant challenges remain: (i) ensuring that the learned representations respect feasibility constraints such as flow conservation and resource bounds, and (ii) aligning the model’s predictions with dual signals and optimality conditions from optimization theory. In this context, combining column generation—a principled approach that iteratively refines feasible regions through subproblem solutions—with GNN-based models trained to approximate those subproblem solutions offers a promising hybrid methodology. By embedding dynamic programming principles, cost-to-go estimation, and constraint satisfaction into the architecture and loss function of the GNN, it becomes possible to learn surrogate functions that guide efficient solution discovery without requiring explicit enumeration. This fusion not only reduces computational burden but also enhances adaptability to problem variations, setting the stage for learning-enhanced solvers that are both scalable and optimization-aligned.

3 Methodology

3.1 Problem Statement

We adopt a column generation approach to solve the vehicle routing problems within scheduling constraints. This approach decomposes the original complex optimization task into a master problem and a series of subproblems with time constraints. The master problem selects optimal subsets of feasible paths, formulated as a set-partitioning problem.

Specifically, let $\mathcal{P}$ denote the set of feasible paths, where each path p has an associated cost c_p . Decision variable y_p indicates whether path p is selected ($y_p = 1$) or not ($y_p = 0$). The original master problem should be a binary integer optimization problem. To provide the dual variables to the subproblem, we relax the binary variable to linear programming. The master problem is defined as follows:

$$\begin{aligned} & \text{minimize} && \sum_{p \in \mathcal{P}} c_p y_p \\ & \text{subject to} && \sum_{p \in \mathcal{P}} a_{ip} y_p = 1, \quad \forall i \in \mathcal{N}, \\ & && y_p \in [0, 1], \quad \forall p \in \mathcal{P}. \end{aligned}$$

In this formulation, each node $\forall i \in \mathcal{N}$ must be covered exactly once by the selected paths. Parameter a_{ip} equals 1 if node i is included in path p , and 0 otherwise.

The subproblems generate feasible paths (columns) by solving constrained shortest path problems, specifically the Elementary Shortest Path Problem with Resource Constraints (ESPPRC). Consider a directed graph $\mathcal{G} \in (\mathcal{V}, \mathcal{E})$, where each edge $(i, j) \in \mathcal{E}$ has an associated cost w_{ij} and t_{ij} , let $w_{ij} = c_{ij} - \pi_i$, where π_i is the dual variable of the restricted master problem. Each subproblem can be formulated as follows:

$$\begin{aligned} & \text{minimize} && \sum_{(i,j) \in \mathcal{E}} w_{ij} x_{ij} \\ & \text{subject to} && \sum_{(i,j) \in \mathcal{E}} x_{ij} - \sum_{(j,i) \in \mathcal{E}} x_{ji} = \begin{cases} 1, & \text{if } i = \text{source}, \\ -1, & \text{if } i = \text{sink}, \\ 0, & \text{otherwise,} \end{cases} \\ & && \sum_{(i,j) \in \mathcal{E}} t_{ij} x_{ij} \leq \mathcal{T}, \\ & && x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in \mathcal{E}. \end{aligned}$$

The solution to each subproblem corresponds to feasible paths that can potentially improve the objective value of the master problem. Paths with negative reduced costs are selected and introduced into the master problem iteratively. Although traditional methods like exact algorithms or heuristic approaches can solve ESPPRCs, they frequently encounter significant computational overhead.

To address this limitation, we propose leveraging the capabilities of Graph Neural Networks (GNNs) to rapidly and efficiently solve these subproblems. Specifically, inspired by recent advancements in GNN-based linear programming frameworks, we model the ESPPRC by relaxing the binary decision variable x_{ij} into probabilistic edge selection variables $p_{ij} \sim \text{Bernoulli}(p)$. Then we define a probabilistic surrogate loss function to present ESPPRC itself, which is differentiable:

$$L_{\text{prob}}(\theta) = \underbrace{\sum_{(i,j) \in \mathcal{E}} w_{ij} p_{ij}}_{\text{Expected value}} + \underbrace{\lambda_{\text{time}} \left(\sum_{(i,j) \in \mathcal{E}} t_{ij} p_{ij} - \mathcal{T} \right)}_{\text{Time Constraint}} + \underbrace{\sum_{i \in \mathcal{V}} \lambda_{\text{flow}} \cdot \varphi(p)}_{\text{Flow Conservation}}$$

where

$$\varphi(p) = \sum_{j: (i,j) \in \mathcal{E}} p_{ij} - \sum_{j: (j,i) \in \mathcal{E}} p_{ji} = q_i \begin{cases} 1, & \text{if } i = \text{source}, \\ -1, & \text{if } i = \text{sink}, \\ 0, & \text{otherwise.} \end{cases}$$

In this formulation, where λ_{time} and λ_{flow} are penalty parameters enforcing resource (time) and flow constraints. Although GNNs typically aggregate information at the node-wise, we formulate this probabilistic surrogate loss function at the edge-wise instead to avoid introducing a bilinear expected value term. We will discuss the transformation from node-wise to edge-wise probabilities in a subsequent part of the report. During implementation, we use the ReLU function to enforce the constraints. *As this project is still on-going, we will provide the shortest path problem (without time constraints) as preliminary experiments and results.

3.2 Model Architecture

Our proposed architecture, *ShortestPathGNN*—*edge*, is designed to approximate the solution to constrained shortest path problems through a differentiable surrogate loss function aligned with dynamic programming (DP). The core idea is to learn a value function over nodes and edge selection probabilities that collectively approximate the Bellman principle of optimality. To achieve this, we build a GNN framework centered around the *MultiHopEdgeGATConv* operator, which performs message passing with a multi-hop edge-aware attention mechanism augmented by recurrent updates.

Each message passing layer in *MultiHopEdgeGATConv* aggregates neighbor information using both edge and node features, with a GRU cell simulating temporal value propagation across multiple hops. This design emulates dynamic programming by allowing node embeddings to evolve iteratively based on cost-to-go values and graph structure, allowing the model to learn relationships between distant nodes, which is essential for solving path planning problems.

3.2.1 Node-Wise Embedding Projection

The input to the network consists of two components: node features $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, edge features $\mathbf{e}_{ij} \in \mathbb{R}^{d_e}$. These features collectively capture topological, structural, and positional information necessary for shortest path reasoning.

Each node’s feature vector $\mathbf{x}_i$ is composed of several interpretable quantities. These include the in-degree and out-degree of the node (to encode local connectivity), two binary indicators marking whether the node is the source or sink of the graph instance, and a 2D positional coordinate (x_i, y_i) to encode the node’s spatial location. Though the edge weights don’t represent the distance between two nodes in the graph, the spatial location helps to learn the graph structure. In addition, we incorporate a low-dimensional spectral encoding derived from the eigenvectors of the directed Laplacian matrix. These spectral features are obtained by computing the smallest nontrivial eigenvectors (excluding the trivial one) of the Laplacian $\mathbf{L} = \mathbf{D}_{\text{out}} - \mathbf{A}$, where $\mathbf{A}$ is the adjacency matrix and $\mathbf{D}_{\text{out}}$ is the out-degree diagonal matrix. The eigenvectors are computed and their real components are retained as feature vectors. These eigenfeatures help capture global graph structure and diffusion properties even in directed graphs.

Each edge (i, j) also carries a rich feature vector $\mathbf{e}_{ij}$. This includes the raw weight w_{ij} , a travel time or delay value t_{ij} , and a categorical indicator $I_{ij} \in \{0, 1\}$, where $I_{ij} = 1$ if the edge has negative cost (i.e., contributes positively to reduced-cost subproblem search), and $I_{ij} = 0$ otherwise. These features are critical for distinguishing cost-improving edges from cost-incurring ones and for guiding the attention mechanism accordingly.

The initial node embeddings are computed by projecting $\mathbf{x}_i$ into a hidden dimension using a linear transformation followed by batch normalization and a ReLU activation:

$$\mathbf{h}_i^{(0)} = \text{ReLU}(\text{BN}(\mathbf{W}_0 \mathbf{x}_i)),$$

where $\mathbf{W}_0 \in \mathbb{R}^{d_{\text{in}} \times d_h}$ is a learnable weight matrix and d_h is the hidden dimension. These node embeddings serve as the input to subsequent message-passing layers that integrate edge and positional information to model dynamic programming transitions across the graph.

3.2.2 Message Passing Layers

The projected embeddings are passed through T stacked *MultiHopEdgeGATConv* layers, each applying multiple internal hops to capture long-range edge dependencies. Each *MultiHopEdgeGATConv* layer performs K internal hops per forward call. The message passing at hop t is defined as:

$$\mathbf{m}_i^{(t)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(t)} \left(\sigma_g(\mathbf{e}_{ij}) \cdot \mathbf{h}_j^{(t)} + \phi_e(\mathbf{e}_{ij}) \right),$$

$$\alpha_{ij}^{(t)} = \text{softmax}_j \left(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{h}_i^{(t)} \parallel \mathbf{h}_j^{(t)} \parallel \mathbf{e}_{ij}]) \right),$$

where:

- $\alpha_{ij}^{(t)}$ is the attention weight;
- σ_g is a sigmoid gate applied to edge features,
- ϕ_e is an MLP used to embed edge features.

The hidden state is then updated via a gated recurrent unit and truncated to ensure numerical stability. This recursive update mimics value propagation in dynamic programming by enforcing monotonic updates and incorporating transition

costs via edge features. After T message passing iterations, each node i holds an embedding $\mathbf{h}_i^{(T)}$ summarizing its cost-aware reachability in the graph:

$$\mathbf{h}_i^{(t+1)} = \min(\mathbf{h}_i^{(t)}, \text{GRU}(\mathbf{m}_i^{(t)}, \mathbf{h}_i^{(t)})).$$

3.2.3 Node-Level Predictions

After the final message passing iteration, each node i obtains a hidden representation $\mathbf{h}_i^{(T)} \in \mathbb{R}^{d_h}$, which encodes structural, positional, and cost-related information propagated across the graph. This representation is used to compute two essential quantities: the node selection probability $p_i \in [0, 1]$ and the node-level value function estimate $v_i \in \mathbb{R}$. Both quantities are generated via separate two-layer multilayer perceptrons (MLPs) applied to the same input embedding $\mathbf{h}_i^{(T)}$.

To compute the node selection probability, the hidden representation is first passed through a linear transformation and a non-linear activation:

$$s_i = \mathbf{w}_2^\top \text{ReLU}(\mathbf{W}_1 \mathbf{h}_i^{(T)} + \mathbf{b}_1) + b_2,$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_1 \times d_h}$ and $\mathbf{b}_1 \in \mathbb{R}^{d_1}$ are the weight and bias of the first linear layer, and $\mathbf{w}_2 \in \mathbb{R}^{d_1}$, $b_2 \in \mathbb{R}$ are the parameters of the second layer. The scalar output $s_i \in \mathbb{R}$ is then passed through a sigmoid activation $\sigma(\cdot)$ to obtain $p_i = \sigma(s_i)$, which represents the model’s estimated probability that node i belongs to the optimal path. The output bias b_2 is initialized using a uniform distribution over $[-2, 2]$ to encourage variance in the initial node probabilities.

Parallel to this, a second MLP predicts the value function for each node. This branch mirrors the structure of the node selection head:

$$v_i = \mathbf{w}_4^\top \text{ReLU}(\mathbf{W}_3 \mathbf{h}_i^{(T)} + \mathbf{b}_3) + b_4,$$

where $\mathbf{W}_3 \in \mathbb{R}^{d_1 \times d_h}$ and $\mathbf{b}_3 \in \mathbb{R}^{d_1}$ are the parameters of the first transformation, and $\mathbf{w}_4 \in \mathbb{R}^{d_1}$, $b_4 \in \mathbb{R}$ are the second-layer parameters. The resulting scalar v_i estimates the expected cost-to-go from node i , and plays a central role in modeling Bellman residuals and soft dynamic programming transitions.

This is a dual-headed design, comprising a probability prediction head and a value estimation head, allowing the network to simultaneously reason about which nodes to choose and how much cost is expected downstream from each node, thereby aligning the node-level outputs with principles from dynamic programming.

3.2.4 Edge-Level Routing Probabilities

We interpret v_i as the cost-to-go estimate from node i , representing the learned estimate of the expected downstream cost starting from that node. Inspired by the dynamic programming (DP) update rule $v_i = \min_j(w_{ij} + v_j)$, we define the conditional edge-selection probability using a differentiable softmin operator. Specifically, given the raw edge cost w_{ij} for edge (i, j) and the value estimate v_j of the destination node j , we first compute the cost-to-go along edge (i, j) as:

$$\text{Value}_i = \min_j (w_{ij} + v_j)$$

Then, the conditional probability of choosing edge (i, j) given node i is defined by applying a softmax operation over the negated costs-to-go for all outgoing edges from node i :

$$\pi_{ij} = \mathbb{P}(Y = j \mid X = i) = \frac{\exp(-\text{Value}_i)}{\sum_{k \in \mathcal{N}^+(i)} \exp(-\text{Value}_{ik})},$$

where $\mathcal{N}^+(i)$ denotes the set of outgoing neighbors of node i . Finally, to account for the probability $p_i = \mathbb{P}(X = i)$ of reaching or being at node i , we obtain the joint probability of using edge (i, j) as:

$$\hat{p}_{ij} = \mathbb{P}(X = i, Y = j) = \mathbb{P}(X = i) \cdot \mathbb{P}(Y = j \mid X = i) = p_i \cdot \pi_{ij} = p_i \cdot \frac{\exp(-(w_{ij} + v_j))}{\sum_{k \in \mathcal{N}^+(i)} \exp(-(w_{ik} + v_k))}.$$

This probabilistic formulation closely mimics the classical DP recurrence, maintains differentiability suitable for gradient-based learning, and provides clear interpretability as a stochastic routing strategy.

3.2.5 Flow Constraints and Source-Sink Vector

To enforce the feasibility of path-based flow, we construct a source-sink vector $\mathbf{q} \in \mathbb{R}^n$ for each graph. For a given subgraph, let s and t be the indices of the source and sink nodes. We define:

$$q_i = \begin{cases} +1, & \text{if } i = s, \\ -1, & \text{if } i = t, \\ 0, & \text{otherwise.} \end{cases}$$

This vector $\mathbf{q}$ enters the flow conservation loss to ensure net inflow and outflow match expectations of unit flow.

3.2.6 Final Loss Function Construction

The total loss is composed of several components reflecting path cost, flow conservation, Bellman consistency, and value update quality:

$$\begin{aligned} \mathcal{L}_{\text{total}} = & \underbrace{\sum_{ij} w_{ij} \hat{p}_{ij}}_{\text{Expected Path Cost}} + \underbrace{\lambda_{\text{flow}} \cdot \sum_i \left(\text{ReLU}(f_i^{\text{out}} - f_i^{\text{in}} - q_i) \right)^2}_{\text{Flow Conservation}} + \underbrace{\lambda_{\text{time}} \cdot \sum_i \left(\text{ReLU}(t_{ij} p_{ij} - \mathcal{T}) \right)}_{\text{time constraint}} \\ & + \underbrace{\lambda_{\text{Bellman}} \cdot \text{MSE}(v_i, \text{softmax}_j(w_{ij} + v_j))}_{\text{Bellman Residual}} + \underbrace{\lambda_{\text{update}} \cdot \mathbb{E}_{(i,j)} [\text{ReLU}(v_i - (w_{ij} + v_j))]}_{\text{Soft Update Error}} \end{aligned}$$

To encourage stable training, the final loss is scaled using a hyperbolic tangent:

$$\mathcal{L}_{\text{final}} = 1000 \cdot \tanh(0.001 \cdot \mathcal{L}_{\text{total}}).$$

3.2.7 Decoding Process

The decoding process aims to extract feasible paths from learned node-level probabilities within a batch of directed graphs, each representing a separate instance of the ESPPRC problem. For each graph in the batch, the method isolates its corresponding subgraph nodes and their associated probabilities, along with designated source and sink nodes. Two decoding strategies are implemented: greedy and sampling-based. The greedy decoder sequentially selects the unvisited neighbor with the highest predicted probability, progressing from source to sink, and halts if no valid next step exists. In contrast, the sampling-based decoder draws multiple candidate paths by probabilistically sampling unvisited successors in proportion to their predicted probabilities, ultimately selecting the path with the minimal accumulated edge weight. This sampling introduces exploration, potentially escaping local optima missed by the greedy strategy. In this report, we show the sampling-based decoding method results.

4 Preliminary Results

This preliminary experiments are designed for the elementary shortest path problem without time constraints. We train a dynamic programming alignment model and a basic model to make a comparison. Basic model including the same node and edge features with the same attention mechanism, however without message passing hops to capture long-range edge dependencies. In the loss function, the basic model contains a probabilistic surrogate loss function (node-wise) to present the elementary shortest path problem itself, but without Bellman residual and soft update error terms to represent the dynamic programming alignment loss. As we could see from the training loss diagram, the Dynamic

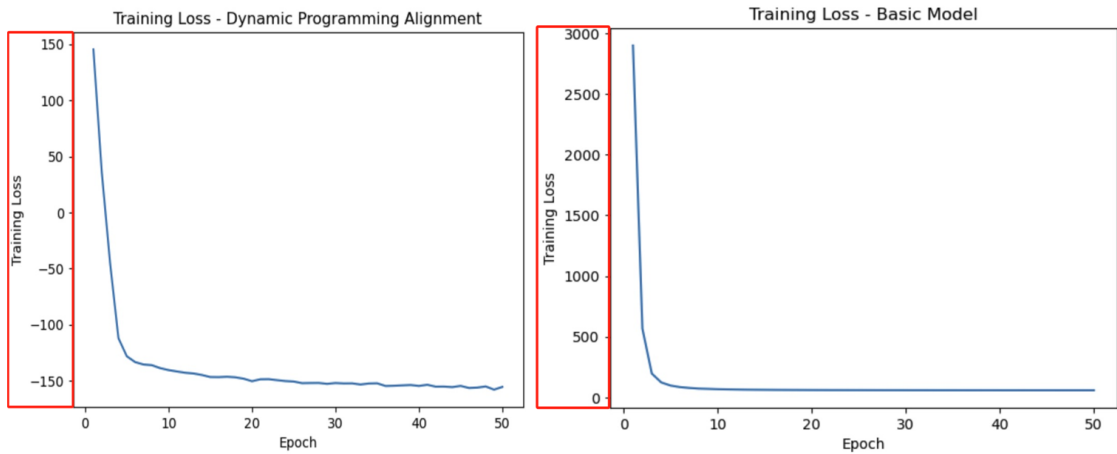


Figure 1: Dynamic Programming Alignment vs. Basic Model

Programming Alignment model a lower final training loss compared to the Basic Model. The evaluation metrics are

defined as follows: Let N be the total number of test graphs, and for each graph $i \in \{1, 2, \dots, N\}$. Correctness Rate measures the percentage of decoded paths whose length exactly matches the ground-truth shortest path length:

$$\text{Correctness Rate} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left(\left| \ell_i^{\text{pred}} - \ell_i^{\text{true}} \right| < \varepsilon \right),$$

Root Mean Square Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\ell_i^{\text{pred}} - \ell_i^{\text{true}} \right)^2}.$$

Average Predicted-to-Total Length Rate measures the efficiency of the predicted path relative to the ground truth of the graph:

$$\text{Mean P-to-T Rate} = \frac{1}{N} \sum_{i=1}^N \frac{\ell_i^{\text{pred}}}{\ell_i^{\text{true}}},$$

$$\text{Negative Length Rate} = \frac{1}{N_{\text{negative}}} \sum_{i=1}^N \mathbf{1}(\ell_i^{\text{pred}} < 0)$$

where ℓ_i^{pred} is predicted path length, ℓ_i^{true} is ground-truth shortest path length, $\mathbf{1}[\cdot]$ is the indicator function (1 if condition is true, 0 otherwise), and ε is numerical tolerance. During implementation, we set $\varepsilon = 10^{-7}$.

Table 1: Comparison of decoding performance between the Dynamic Programming Model and the Basic Model.

Model	Correctness Rate	RMSE	Mean P-to-T Rate	Negative Length Rate
Dynamic Programming Model	47.5% (95/200)	1.7397	74.4090%	90.71% (166/183)
Basic Model	16.5% (33/200)	3.7975	31.0664%	57.92% (106/183)

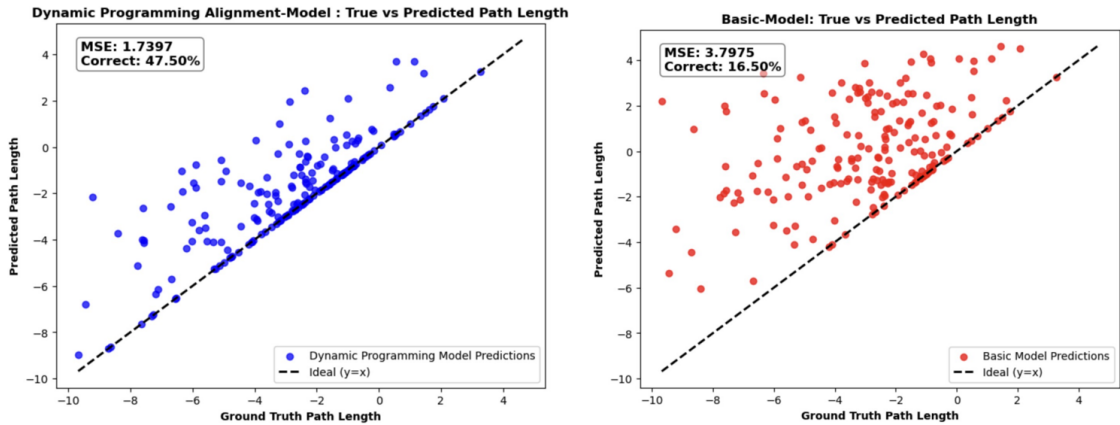


Figure 2: Dynamic Programming Alignment vs. Basic Scatter Plot

Specifically, we also investigated the dynamic programming model under different sampling numbers: The results demonstrate the effectiveness of our Dynamic Programming (DP) model in decoding elementary shortest paths. As shown in Table 1, the DP model significantly outperforms the Basic Model across all evaluation metrics: achieving a correctness rate of 47.5% versus 16.5%, a lower RMSE of 1.7397 compared to 3.7975, and a notably higher mean predicted-to-true length ratio of 74.4090% versus 31.0664%. Furthermore, the DP model maintains a high negative-length path recovery rate of 90.71%, indicating its potential in generating high-quality solutions even in challenging graph structures. Figure 1 further supports these findings by visualizing the close alignment between the predicted and true path lengths under the DP model, in contrast to the more scattered predictions of the Basic Model. Additionally, as shown in Table 2, increasing the sampling number M steadily improves performance, with correctness rate rising to 71.0% and RMSE decreasing to 0.7793 at $M = 5000$.

Table 2: DP MODEL:Decoding performance under different sampling numbers M .

Sampling Number M	Correctness Rate	RMSE	Mean P-to-T Rate	Negative Length Rate
50	36.5% (73/200)	2.1183	60.0062%	85.25% (156/183)
100	47.5% (95/200)	1.7397	74.4090%	90.71% (166/183)
200	52.0% (104/200)	1.4939	75.5433%	91.26% (167/183)
500	54.5% (109/200)	1.3749	82.1843%	94.54% (173/183)
1000	62.0% (124/200)	1.1717	82.5652%	96.17% (176/183)
5000	71.00% (142/200)	0.7793	91.5211%	98.91% (181/183)

We also draw the boxplot for each graph Monte Carlo sampling results based on our GNN output probabilities. Figure 3 presents the distribution of sampled path lengths generated by the Dynamic Programming model for the first 20 test graphs. Each box plot corresponds to one graph and shows the variation in path lengths from 100 sampled paths decoded using the model’s predicted node probabilities. The red star markers indicate the ground-truth shortest path lengths obtained via the Bellman-Ford algorithm. The close alignment between the ground truth and the lower end of the sampled distributions in many cases demonstrates the effectiveness of the Dynamic Programming model in guiding the sampling toward near-optimal paths. quality.

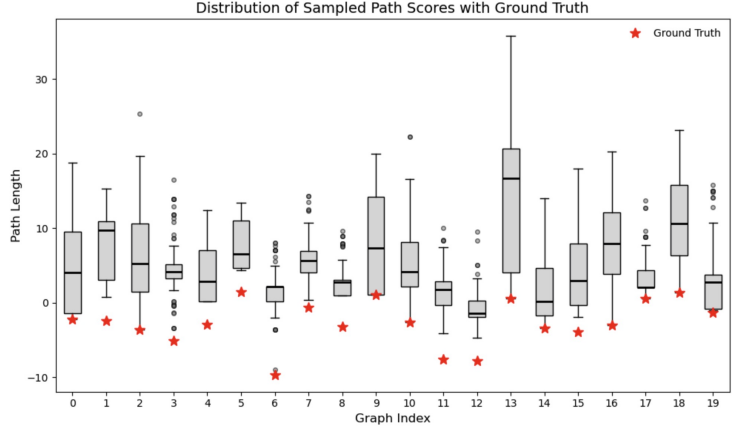


Figure 3: DP Model: Distribution of Sampled Path Length with Ground Truth. This figure provides a detailed per-graph view of the model’s path

5 Discussion and Conclusion

This work introduces a learning-based optimization framework that integrates GNN and column generation together to approximate solutions for constrained vehicle routing problems, especially ESPPRC. We leverage a differentiable surrogate loss function based on dynamic programming principles to guide the approximation of paths with less cost. With the combination of multi-hop message passing, flow conservation constraints, and Bellman residual regularization, our proposed model is able to learn the approximation of cost-to-go values and edge selection probabilities when given the constraints to find a cost-effective path.

The experiment results of synthetic graphs illustrate the effectiveness of our DP alignment model over a baseline in capturing the feature of shortest paths. The higher correctness rate and lower RMSE of the combination with different decoding strategies imply the robustness of our methods for recovering the shortest paths without clear enumeration. Besides, the increase in decoding accuracy with the number of samples further validates the quality of probabilistic modeling.

Several directions could be explored in the future. First, our current experiments are conducted on simplified graphs without incorporating time or capacity constraints. Extending the model to handle fully constrained ESPPRC instances, including time windows, precedence, and resource bounds, is a critical next step. Second, despite our model generates well in the synthetic graphs, we still need to validate the effectiveness in real-world transportation road networks and applications, which involves a large amount of noise and irregular topological structures. Grounding our model in benchmark datasets could provide stronger empirical evidence of its practical utility.

Moreover, although the appearance of a surrogate loss function enables a differentiable objective function, the relaxation may affect the quality of the solution in certain cases. Ways of incorporating constraint-aware post-processing will be explored in future work to improve the robustness of the framework.

Still, we believe that this work is bridging the gap between combinatorial optimization problems and machine learning solutions. By combining classical algorithmic DP principles with neural network structures, and aligning training objectives with signal from traditional solvers, learning-based methods can handle more complex constraints and become more efficient. Deploying it into operational transportation applications will be essential to fully evaluate and realize its potential impact.

References

- [1] Yossiri Adulyasak and Patrick Jaillet. Models and algorithms for stochastic and robust vehicle routing with deadlines. *Transportation Science*, 50(2):608–626, 2016.
- [2] Yiyong Xiao and Abdullah Konak. The heterogeneous green vehicle routing and scheduling problem with time-varying traffic congestion. *Transportation Research Part E: Logistics and Transportation Review*, 88:146–166, 2016.
- [3] A Kazhaev, Z Almetova, V Shepelev, and K Shubenkova. Modelling urban route transport network parameters with traffic, demand and infrastructural limitations being considered. In *IOP Conference Series: Earth and Environmental Science*, volume 177, page 012018. IOP Publishing, 2018.
- [4] Hai Wang. Routing and scheduling for a last-mile transportation system. *Transportation Science*, 53(1):131–147, 2019.
- [5] Carlos F Daganzo and Yanfeng Ouyang. A general model of demand-responsive transportation services: From taxi to ridesharing to dial-a-ride. *Transportation Research Part B: Methodological*, 126:213–224, 2019.
- [6] Tomislav Erdelić and Tonči Carić. A survey on the electric vehicle routing problem: variants and solution approaches. *Journal of Advanced Transportation*, 2019(1):5075671, 2019.
- [7] Laurence A Wolsey and George L Nemhauser. *Integer and combinatorial optimization*. John Wiley & Sons, 1999.
- [8] Quentin Cappart, Didier Chételat, Elias B Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023.
- [9] Christos H Papadimitriou and Kenneth Steiglitz. *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.
- [10] Stefan Irnich and Guy Desaulniers. Shortest path problems with resource constraints. In *Column generation*, pages 33–65. Springer, 2005.
- [11] Paolo Brandimarte. *From shortest paths to reinforcement learning*. Springer, 2021.
- [12] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. *Advances in neural information processing systems*, 30, 2017.
- [13] Bernhard H Korte, Jens Vygen, B Korte, and J Vygen. *Combinatorial optimization*, volume 1. Springer, 2011.
- [14] Kairanbay Magzhan and Hajar Mat Jani. A review and evaluations of shortest path algorithms. *Int. J. Sci. Technol. Res.*, 2(6):99–104, 2013.
- [15] Leonardo Lozano, Daniel Duque, and Andrés L Medaglia. An exact algorithm for the elementary shortest path problem with resource constraints. *Transportation Science*, 50(1):348–357, 2016.
- [16] Dominique Feillet, Pierre Dejax, Michel Gendreau, and Cyrille Gueguen. An exact algorithm for the elementary shortest path problem with resource constraints: Application to some vehicle routing problems. *Networks: An International Journal*, 44(3):216–229, 2004.
- [17] Giovanni Righini and Matteo Salani. New dynamic programming algorithms for the resource constrained elementary shortest path problem. *Networks: An International Journal*, 51(3):155–170, 2008.
- [18] Nikolaos Karalias and Andreas Loukas. Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33:6659–6672, 2020.
- [19] Dingyi Zeng, Wanlong Liu, Wenyu Chen, Li Zhou, Malu Zhang, and Hong Qu. Substructure aware graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 11129–11137, 2023.